ECE 4301 Quiz 2 Report

Joshua Deckman and Jack Pearson

Setup and Methods

For this second quiz, both the SHA-1 and SHA-256 hashing algorithms were implemented in Rust on a Raspberry Pi 5. By adjusting build flags, the implementation of these algorithms was compiled into two different executables—one to use hardware acceleration provided by the Pi and the other to solely rely on software implementations of the algorithms. Both the executable with hardware acceleration and the one that relied only on software were benchmarked in a series of tests. Each test presented a random buffer of bytes as an input to each executable and then timed their execution. To obtain accurate results, each hashing algorithm was run ten times on each input buffer. Once the results from these tests were acquired, the throughput of each algorithm for both executables was calculated. The lengths of the input byte buffers that were used for the tests were 1 KiB, 8 KiB, 64 KiB, and 1 MiB.

Once these tests were run, similar tests (with the same byte buffers) were performed on the Raspberry Pi's built-in kernel hashing engine. Its throughput was also measured and compared to that of the other implementations.

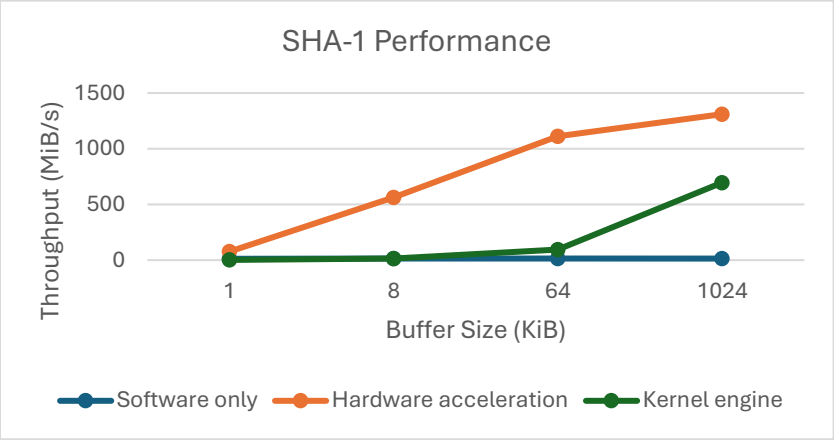
After these tests were conducted, a demonstration of the project was made. This demonstration used a webcam, which was connected to the Pi, to take pictures. An image of the setup follows. As pictures were taken by the Pi, special code took the pictures and used the two executables to calculate their SHA-1 and SHA-256 hashes.

Raspberry Pi 5 and Webcam

Plots

SHA-1 Experiment Results

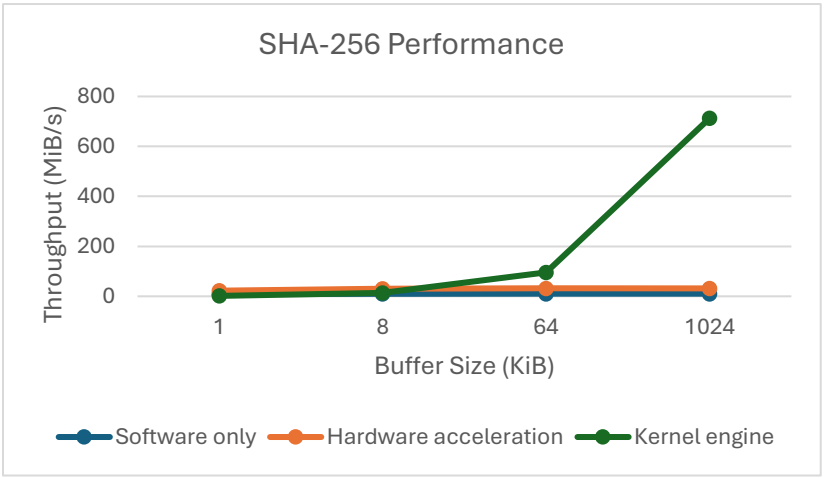
Input Buffer Size (KiB)	Throughput (MiB/s)		
	Software only	Hardware acceleration	Kernel engine
1	11.87	76.42	1.67
8	13.55	563.7	14.16
64	13.83	1112.07	96.06
1024	13.93	1310.46	695.99



Plot of SHA-1Throughput vs. Size by Method

SHA-256 Experiment Results

Input Buffer Size (KiB)	Throughput (MiB/s)		
	Software only	Hardware acceleration	Kernel engine
1	8.88	22.64	1.68
8	10.13	30.39	14.18
64	10.42	32.23	95.8
1024	10.42	32.08	712.91



Plot of SHA-256 Throughput vs. Size by Method

Analysis and Recommendations

According to the data acquired in the experiments, software-only hashing is always slower than hashing with hardware acceleration, indicating that the Armv8 SHA extended instruction set does provide some benefit to hashing calculations. In the case of the SHA-1 algorithm, the hardware-accelerated executable is always faster than both the software-only implementation and the kernel engine. In fact, the throughput of the hardware-accelerated version grows rapidly with the size of the input buffer until it hits a plateau between input buffer sizes of 64 KiB and 1 MiB. However, in the case of SHA-256, its plateau comes much sooner, being between buffer sizes of 8 KiB and 64 KiB. Notably, the performance of the hardware-accelerated executable in the case of SHA-256 is substantially inferior to that of SHA-1. Although the accelerated version can still calculate the SHA-256 hash faster than the software-only version, it is not clear that the hardware is aiding in this calculation as much as it is for SHA-1.

In contrast to the hardware-accelerated hashing implementation, the speed of the kernel engine is fairly consistent between SHA-1 and SHA-256. In both cases, the throughput of the kernel engine is fairly similar, growing rapidly with respect to input buffer size. Although the kernel implementation is not quite as fast as the hardware-accelerated version in the case of SHA-1, it is significantly faster than the hardware-accelerated implementation of SHA-256. Further, unlike the hardware-accelerated version, the kernel engine does not appear to have a plateau over the buffer sizes that were used in the experiments. Consequently, the kernel engine has a significant advantage over the hardware-accelerated implementation of the SHA-256 hashing algorithm.

It should be noted that, for small input buffer sizes (1 KiB in these experiments), the kernel engine is slower than both the hardware-accelerated and the software-only implementations of the hashing algorithms. This is likely due to overhead associated with initiating communication with the kernel engine. However, as the size of the input buffer increases, this overhead becomes insignificant.

As shown in the graphs, the greatest throughput of the hashing algorithms occurs when large buffers are passed to them. Thus, in the case of sending telemetry data, large-buffer telemetry appears to be much more efficient than sending many small packets of data. If the SHA-1 algorithm is to be used with this type of telemetry, then it is clear that hardware acceleration should be used. On the other hand, if the SHA-256 hash is to be used, then the Raspberry Pi's kernel engine is faster than the other implementations.